

Sliding Windows and Prefix Sums

Contiguous-range problems often become efficient when you maintain only the state that changes at the boundaries. Sliding windows use a safe expand/shrink rule; prefix sums turn a range relationship into a relationship between earlier states. The input constraints determine which reasoning applies.

For: Candidates who recognize subarray and substring questions but need to distinguish fixed windows, variable windows, and prefix-state methods correctly.

Suggested pace: 10-14 focused practice sessions | Arrays and maps understood

What you will be able to do

- Define window validity and update it correctly as boundaries move.
- Recognize when negative values or exact-count requirements break a simple window.
- Use prefix-frequency maps to count all matching ranges.
- Test off-by-one, empty-prefix, and duplicate-prefix cases deliberately.

Build the right mental model

Fixed windows update two boundaries

For a window of length k , add the incoming value and remove the outgoing one. Carefully define when the first full window exists. For character problems, compare multiplicities, not only distinct membership.

Variable windows need monotonicity

A window can shrink greedily only when the validity rule supports it. Positive sums and at-most- k distinct counts have useful monotone behavior; arbitrary negative sums do not follow the same rule. State what remains true after every boundary update.

Prefixes encode history

If current prefix sum is S , a subarray of target sum T ends here for every earlier prefix equal to $S-T$. Initialize the empty prefix once. Store counts when several earlier prefixes can produce distinct ranges.

Study with evidence.

Draw indexes and window state for a short example before coding. Use separate exercises for maximum/minimum length and number of matching ranges; they often need different retained information. Explain why the left boundary never moves backward when that is your invariant.

Sessions 1-3: Fixed-size windows

Practice rolling sums and anagram windows. Validate k and input length, update incoming and outgoing counts, and decide whether the result is a value, index, or list of windows.

Deliverable and check: A trace showing the first valid window and every subsequent update without recomputing the whole range.

Sessions 4-6: Variable-size windows

Study unique-character windows, replacement budgets, and minimum positive-sum windows. Define the validity predicate and the reason shrinking is safe. Test repeated characters near the current left boundary.

Deliverable and check: A proof that each index is advanced at most a bounded number of times and no valid candidate is skipped.

Sessions 7-10: Prefix sums and counts

Practice exact subarray sums with zero and negative values. Compare storing counts, earliest positions, and all positions. Initialize the empty prefix and avoid inserting the current prefix too early for the intended contract.

Deliverable and check: Worked examples with repeated prefix values and a clear count of distinct subarrays.

Sessions 11-14: Mixed decisions

Remove topic labels and solve variants involving circular arrays, bounds, and exact versus at-most constraints. Explain a counterexample for an inappropriate window approach before selecting an alternative.

Deliverable and check: An unseen mixed problem solved with a justified choice and boundary tests.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Count zero-sum subarrays with repeated prefixes

For values $[1, -1, 1, -1]$ and target zero, count every contiguous range whose sum is zero. Negative values make a simple positive-sum shrinking window unreliable.

A worked approach

1. Start with prefix frequency $\{0: 1\}$, representing the empty prefix. After reading 1, the current sum is 1 and there are no earlier prefixes with that sum; then record frequency 1 for prefix 1.
2. After -1, the current sum is 0 and one earlier zero prefix contributes one range. Record zero frequency 2. At the next 1, one earlier prefix 1 contributes another range; record frequency 2 for prefix 1.
3. At the final -1, the current sum is zero and two earlier zero prefixes contribute two more ranges. The total is four. Looking up before recording avoids counting an empty range at the current position.

Expected result

Four zero-sum subarrays are counted in expected $O(n)$ time using prefix frequencies. The map stores multiplicity because distinct earlier boundaries can have the same sum.

Verify the result

- Does an all-zero array of length three produce six nonempty zero-sum ranges?
- Is the empty prefix initialized exactly once?
- Does the implementation distinguish counts from a single remembered index?

Extend the exercise

Ask for the longest zero-sum range instead. Store the earliest index for each prefix sum and explain why later duplicate indexes should not replace it when maximizing length.

Make the learning stick.

Why not use sliding windows for every sum problem?

The window must have a monotone relationship between boundary changes and validity. Negative values can make extension reduce the sum or removal increase it, invalidating the simple positive-input argument.

Why is the initial zero prefix necessary?

It represents ranges that start at index zero. Without it, a prefix equal to the target has no earlier boundary in the map and is incorrectly omitted.

When do I store earliest versus latest indexes?

It depends on the objective. Earliest positions often maximize length; latest can minimize it under the appropriate contract. Counts are needed when all matching ranges contribute. Choose state from the requested result, not from a memorized template.

Avoid these traps

- Applying a positive-sum window to arbitrary signed values.
- Moving the left boundary backward after a repeated character.
- Replacing frequency counts with a set and losing valid ranges.

Ready for the next step?

- Window validity and update order are explicit.
- Counterexamples distinguish windows from prefix methods.
- Zero and repeated-prefix cases are covered.
- The stored map state matches the optimization or counting goal.

Reliable next reads

- [MIT algorithms course](#)
- [Python collections](#)
- [NeetCode roadmap](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.